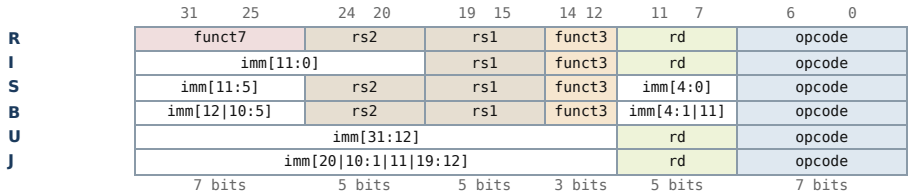
RV32I — RISC-V 32-bit Base Integer ISA

One-page encoding and instruction reference · unprivileged base integer instruction set

Revision 1.0 · September 2026
Public domain (CC0 1.0) — no rights reserved

INSTRUCTION FORMATS — every base instruction is exactly 32 bits wide



Fixed field placement. `rs1`, `rs2` and `rd` always sit in the same bit positions, so the register file can be read before the opcode is decoded.

Sign bit anchored. Bit 31 is the sign bit of every immediate, which keeps sign-extension off the critical path. S/B and U/J differ only in how the remaining immediate bits are scattered.

NOTATION USED IN THIS CARD

`x[n]`	integer register *n*; `x[0]` always reads 0 and discards writes
`pc`	program counter of the instruction being executed
`M[a]`	memory byte at address *a*; subscript range selects the accessed bits
`imm`	immediate field, assembled from the scattered bits shown at left
`shamt`	shift amount, `imm[4:0]` (0–31) for the immediate shifts
`sx(v)`	sign-extend *v* to 32 bits
`zx(v)`	zero-extend *v* to 32 bits
`ea`	effective address, `x[rs1] + sx(imm)`
$<_s$	signed (two's-complement) comparison
$<_u$	unsigned comparison
$>>_a$	arithmetic right shift (sign-replicating)
`←`	assignment, performed at the end of the instruction
`<< 12`	value placed in bits 31:12 with zeros below

Reading the table. The *opcode* column gives bits 6:0 in binary with the hexadecimal value in parentheses; *f3* is bits 14:12 and *f7* is bits 31:25. A dash means the field is not used to select the instruction.

RV32I BASE INTEGER INSTRUCTIONS — 37 instructions, grouped by function

opcode	f3	f7	fmt	assembly syntax	meaning	operation
LOADS — READ MEMORY INTO A REGISTER						
0000011 (03)	000	–	I	**lb** rd, imm(rs1)	load signed byte	x[rd] ← sx(M[ea]7:0)
0000011 (03)	001	–	I	**lh** rd, imm(rs1)	load signed halfword	x[rd] ← sx(M[ea]15:0)
0000011 (03)	010	–	I	**lw** rd, imm(rs1)	load word	x[rd] ← M[ea]31:0
0000011 (03)	100	–	I	**lbu** rd, imm(rs1)	load unsigned byte	x[rd] ← zx(M[ea]7:0)
0000011 (03)	101	–	I	**lhu** rd, imm(rs1)	load unsigned halfword	x[rd] ← zx(M[ea]15:0)
STORES — WRITE A REGISTER TO MEMORY						
0100011 (23)	000	–	S	**sb** rs2, imm(rs1)	store byte	M[ea]7:0 ← x[rs2]7:0
0100011 (23)	001	–	S	**sh** rs2, imm(rs1)	store halfword	M[ea]15:0 ← x[rs2]15:0
0100011 (23)	010	–	S	**sw** rs2, imm(rs1)	store word	M[ea]31:0 ← x[rs2]
REGISTER – IMMEDIATE ARITHMETIC AND LOGIC						
0010011 (13)	000	–	I	**addi** rd, rs1, imm	add immediate	x[rd] ← x[rs1] + sx(imm)
0010011 (13)	010	–	I	**slti** rd, rs1, imm	set if less, signed	x[rd] ← (x[rs1] $<_s$ sx(imm)) ? 1 : 0
0010011 (13)	011	–	I	**sltiu** rd, rs1, imm	set if less, unsigned	x[rd] ← (x[rs1] $<_u$ sx(imm)) ? 1 : 0
0010011 (13)	100	–	I	**xori** rd, rs1, imm	bitwise xor immediate	x[rd] ← x[rs1] ^ sx(imm)
0010011 (13)	110	–	I	**ori** rd, rs1, imm	bitwise or immediate	x[rd] ← x[rs1] | sx(imm)
0010011 (13)	111	–	I	**andi** rd, rs1, imm	bitwise and immediate	x[rd] ← x[rs1] & sx(imm)
0010011 (13)	001	0000000	I	**slli** rd, rs1, shamt	shift left logical imm.	x[rd] ← x[rs1] << shamt
0010011 (13)	101	0000000	I	**srli** rd, rs1, shamt	shift right logical imm.	x[rd] ← x[rs1] >> shamt
0010011 (13)	101	0100000	I	**srai** rd, rs1, shamt	shift right arith. imm.	x[rd] ← x[rs1] $>>_a$ shamt
REGISTER – REGISTER ARITHMETIC AND LOGIC						
0110011 (33)	000	0000000	R	**add** rd, rs1, rs2	add	x[rd] ← x[rs1] + x[rs2]
0110011 (33)	000	0100000	R	**sub** rd, rs1, rs2	subtract	x[rd] ← x[rs1] − x[rs2]
0110011 (33)	001	0000000	R	**sll** rd, rs1, rs2	shift left logical	x[rd] ← x[rs1] << x[rs2]4:0
0110011 (33)	010	0000000	R	**slt** rd, rs1, rs2	set if less, signed	x[rd] ← (x[rs1] $<_s$ x[rs2]) ? 1 : 0
0110011 (33)	011	0000000	R	**sltu** rd, rs1, rs2	set if less, unsigned	x[rd] ← (x[rs1] $<_u$ x[rs2]) ? 1 : 0
0110011 (33)	100	0000000	R	**xor** rd, rs1, rs2	bitwise xor	x[rd] ← x[rs1] ^ x[rs2]
0110011 (33)	101	0000000	R	**srl** rd, rs1, rs2	shift right logical	x[rd] ← x[rs1] >> x[rs2]4:0
0110011 (33)	101	0100000	R	**sra** rd, rs1, rs2	shift right arithmetic	x[rd] ← x[rs1] $>>_a$ x[rs2]4:0
0110011 (33)	110	0000000	R	**or** rd, rs1, rs2	bitwise or	x[rd] ← x[rs1] | x[rs2]
0110011 (33)	111	0000000	R	**and** rd, rs1, rs2	bitwise and	x[rd] ← x[rs1] & x[rs2]
UPPER IMMEDIATES — BUILD 32-BIT CONSTANTS AND ADDRESSES						
0110111 (37)	–	–	U	**lui** rd, imm	load upper immediate	x[rd] ← imm31:12 << 12
0010111 (17)	–	–	U	**auipc** rd, imm	add upper imm. to pc	x[rd] ← pc + (imm31:12 << 12)
CONDITIONAL BRANCHES — OFFSET IS A MULTIPLE OF 2, RANGE ±4 KIB						
1100011 (63)	000	–	B	**beq** rs1, rs2, label	branch if equal	if x[rs1] = x[rs2] → pc ← pc + sx(imm)
1100011 (63)	001	–	B	**bne** rs1, rs2, label	branch if not equal	if x[rs1] ≠ x[rs2] → pc ← pc + sx(imm)
1100011 (63)	100	–	B	**blt** rs1, rs2, label	branch if less, signed	if x[rs1] $<_s$ x[rs2] → pc ← pc + sx(imm)
1100011 (63)	101	–	B	**bge** rs1, rs2, label	branch if ≥, signed	if x[rs1] $\geq_s$ x[rs2] → pc ← pc + sx(imm)
1100011 (63)	110	–	B	**bltu** rs1, rs2, label	branch if less, unsigned	if x[rs1] $<_u$ x[rs2] → pc ← pc + sx(imm)
1100011 (63)	111	–	B	**bgeu** rs1, rs2, label	branch if ≥, unsigned	if x[rs1] $\geq_u$ x[rs2] → pc ← pc + sx(imm)
UNCONDITIONAL JUMPS — THE RETURN ADDRESS IS WRITTEN TO RD						
1101111 (6F)	–	–	J	**jal** rd, label	jump and link (±1 MiB)	x[rd] ← pc + 4; pc ← pc + sx(imm)
1100111 (67)	000	–	I	**jalr** rd, imm(rs1)	jump and link register	t ← pc + 4; pc ← (x[rs1] + sx(imm)) & ~1; x[rd] ← t

Register file. 32 registers `x0`–`x31`, each 32 bits wide. `x0` always reads as zero and ignores writes.

Memory. Byte-addressed and little-endian; instructions are 4-byte aligned.

Immediates. Every immediate is sign-extended from bit 31 of the instruction, so the sign bit is always in the same place.

Branch and jump targets. Offsets are relative to the address of the branch or jump itself, and are always even.

Shifts. Only the low 5 bits of the shift amount are used, whether it comes from an immediate or from `rs2`.

Beyond the base. Multiply/divide, atomics, floating point and compressed encodings are separate extensions.

Public domain. To the extent possible under law, the author has waived all copyright and related rights to this document under CC0 1.0. Copy it, print it, modify it, redistribute it — attribution is not required.

Encodings follow the open RISC-V unprivileged ISA specification. Originally written reference — no third-party text or layout reproduced.